8. Hafta

Veritabanı: SQLAlchemy ve Kalıcılık

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- SQLAlchemy 2.x bildirimsel modellerini (`DeclarativeBase`, `Mapped`, `mapped_column`) tanımlayarak, bir `engine`/`Session`/`sessionmaker` kurup `bilet.db` SQLite dosyasında `metadata.create_all` ile tablo oluşturabilecek; ORM modeli (`EtkinlikDB`) ile Pydantic modelini (`Etkinlik`) ayırt edip `model_validate(from_attributes=True)` köprüsünü kullanabileceksiniz;
- `ForeignKey` ve `relationship`/`back_populates` ile Etkinlik 1–N Koltuk 1–0..1 Bilet ilişkilerini kurabilecek; tembel (lazy) ve istekli (`selectinload`) yükleme stratejilerini ayırt edip N+1 sorununu tanıyabilecek; etkinlik silme için cascade kuralını (satılmış bileti olan koltuk varsa `409`, yoksa kademeli silme) uygulayabileceksiniz;
- `select()`/`where()`/`order_by()`/`limit()`/ `offset()` ile sayfalanmış sorgular yazabilecek, `func.count` ile toplam kayıt sayısını hesaplayabilecek, `echo=True` ile üretilen SQL'i okuyup YG II'deki elle yazdığınız SQL ile karşılaştırabilecek; gerektiğinde `text()` ile ham bir SQL sorgusu çalıştırabileceksiniz;
- Bir `Session` içinde `commit`/`rollback` disiplinini uygulayarak, aynı koltuğa eşzamanlı bir rezervasyon denemesinin ikinci isteğinin bir veritabanı bütünlük hatasıyla (`IntegrityError`) karşılaşıp `409`'a çevrilmesini sağlayabilecek; `pytest` ile bir transaction testinizi yazabileceksiniz;
- Bu becerileri birleştirip Hf.6-7'nin API sözleşmesini hiç değiştirmeden, deposunu bellekten SQLite'a taşıyarak Etkinlik/Bilet Sistemine kalıcılık ve gerçek eşzamanlı rezervasyon koruması ekleyebileceksiniz.

Hf.6-7 gerçek bir sunucu ve gerçek bir ön yüz kurdu, ama ikisinin de "hafızası" hâlâ geçiciydi — sunucu süreci her yeniden başladığında (`uvicorn --reload` dahil) tüm etkinlik/koltuk/bilet verisi kayboluyordu. Bu hafta bu veri, SQLAlchemy ORM aracılığıyla bir SQLite dosyasına (`bilet.db`) yazılır ve kalıcı hâle gelir. Ayrıca Hf.6'nın "aynı koltuğu iki kullanıcı aynı anda alamaz" iş kuralı (`409 Conflict`), o zamana kadar yalnızca bir şans eseriydi (tek Python süreci istekleri sırayla işliyordu) — bu hafta bu kural gerçek bir veritabanı transaction'ıyla güvence altına alınır.

Dört ünite boyunca sırasıyla SQLAlchemy'nin bildirimsel model biçimini,

`engine/Session/get_db` desenini ve ORM↔ Pydantic köprüsünü (Ünite 1); Etkinlik–Koltuk–Bilet arasındaki ilişkileri, tembel/istekli yüklemeyi ve etkinlik silmenin cascade kuralını (Ünite 2); sayfalanmış sorguları, `echo=True` ile ORM'in ürettiği gerçek SQL'i ve `text()` kaçış kapısını (Ünite 3); son olarak bir rezervasyonun tek bir transaction'da nasıl korunduğunu, SQLite'ın yazma kilidini ve bir bütünlük kısıtının son güvence olarak nasıl çalıştığını (Ünite 4) göreceksiniz.

Bu haftanın da bilinçli sınırları vardır. `istemci/` klasörü bu hafta hiç değişmez — Hf.7'nin birebir kopyasıdır; ön yüz sunucunun içinde ne olduğunu bilmez. Sunucu tarafında rotalar ve Pydantic modelleri de mümkün olduğunca değişmeden kalır — tek gerçek değişiklik `app/depo.py`'nin içinin bellekten (dict) veritabanına (SQLAlchemy/SQLite) taşınmasıdır. Alembic bu hafta kurulmaz, yalnızca adı anılır — şema göçlerinin sürümlenmesi bu kitabın kapsamı dışındadır. Postgres kapsam dışındadır, yalnız SQLite kullanılır. Kimlik doğrulama/`Kullanici` tablosu yoktur — bu Hf.10'un konusudur; `kullanici_adi` alanı hâlâ düz metindir. N+1 sorunu yalnızca kısa tanıtılır — performans olarak derinlemesine ele alınması Hf.15'in konusudur.

8.1 ORM Kavramı, Model-Tablo Eşlemesi ve Oturum

1 • Ön-öğretim — Neden Önemli?

Hf.7'de React uygulamanız artık gerçek sunucudan veri çekiyor, ama sunucunun kendi hafızası hâlâ geçici — her `uvicorn --reload` yeniden başladığında girdiğiniz tüm etkinlik/koltuk/bilet verisi sıfırlanıyor. Dönem projeniz için "her ders başında verileri yeniden girme" kabul edilemez; bu ünitede bu veriyi bir dosyaya yazmanın yolunu öğreneceksiniz.

1 • Ön-öğretim — Ön Bilgi

YG II'nin SQLite deneyimini hatırlayın: elle `CREATE TABLE`, `INSERT INTO`, `SELECT ... WHERE` yazıp dönen satırları kendi elinizle bir yapıya/nesneye çevirdiniz. Hf.6 Ünite 2'nin Pydantic `BaseModel` bilgisi bu ünitede ORM modeliyle karşılaştırılacak; Hf.3 Ünite 3'ün `Etkinlik`/`Koltuk`/`Bilet` alan tipleri bu ünitenin ORM sınıflarının alan adlarının birebir karşılığıdır.

Bir önceki haftalarda `Depo` sınıfınız bir Python `dict` üzerinde çalışıyordu — basit, ama sunucu her kapandığında sıfırlanan bir depo. Bu ünitede önce bu tekrar eden "satırı nesneye çevirme" işini kimin, nasıl otomatikleştirdiğini göreceksiniz; ardından bu otomasyonu bir SQLite dosyasına bağlayacak, son olarak bir isteğin veritabanıyla nasıl "konuştuğunu" (oturum yaşam döngüsü) izleyeceksiniz.

8.1.1 Neden ORM? Tekrar eden çeviri işini otomatikleştirmek

YG II'de bir tabloyla çalışırken üç yeri ayrı ayrı güncellemeniz gerekiyordu: tabloyu tanımlayan `CREATE TABLE` cümlesi, dönen satırları bir yapıya çeviren kod ve o yapıyı geri veritabanına yazan kod. Bir sütun eklediğinizde bu üç yerin her birinin güncellenmesi gerekiyordu — unutulması kolay, hataya açık bir iştir. Bir **ORM (nesne-ilişkisel eşleme)**, bu üç işi tek bir yerde, bir Python sınıfının tanımında birleştirir: sınıfı tanımladığınızda hem tablo şeması hem satır↔nesne çeviri mantığı otomatik olarak ortaya çıkar.

ORM (nesne-ilişkisel eşleme): Python sınıflarını veritabanı tablolarına, nesneleri satırlara eşleyen katman (*object-relational mapping*).

Analoji

ORM'i bir tercümana benzetin: Python nesneleriniz Python "konuşur", veritabanı SQL "konuşur"; ORM ikisi arasında çeviri yapan bir tercümandır — siz Python nesnesiyle konuşursunuz, tercüman arka planda doğru SQL cümlesini kurar ve karşı tarafın (veritabanının) cevabını yine sizin dilinize çevirir.

8.1.2 SQLAlchemy 2.x bildirimsel modeli

bildirimsel model: Sınıf tanımından tablo şemasının otomatik türetildiği model biçimi (*declarative model*).

SQLAlchemy 2.x'in **bildirimsel model**i şu üç parçadan oluşur: tüm modellerin miras aldığı bir taban sınıf, her modelin `__tablename__` ile tablo adını belirtmesi ve her alanın `Mapped[tip]` tip ipucu + `mapped_column(...)` çağrısıyla tanımlanması. Taban sınıf şöyle kurulur:

```
class Base(DeclarativeBase):
    ...
```

Bu satırlar Hf.6 Ünite 2'nin `class EtkinlikOlustur(KamelModel)` tanımına çok benzer görünür — ikisi de bir sınıf tanımından bir şey üretir. Ama ürettikleri farklıdır: Pydantic'in `BaseModel`'i tip ipucundan bir doğrulama kuralı üretirken, SQLAlchemy'nin `Base`'i tip ipucundan bir veritabanı sütun tanımı üretir. `EtkinlikDB` modeli şöyle tanımlanır:

```
class EtkinlikDB(Base):

    __tablename__ = "etkinlikler"

    id: Mapped[int] = mapped_column(primary_key=True)
    baslik: Mapped[str]
    tarih: Mapped[str]
    mekan: Mapped[str]
    kapasite: Mapped[int]
    bilet_fiyati: Mapped[float]

    koltuklar: Mapped[list["KoltukDB"]] = relationship(
        back_populates="etkinlik", cascade="all, delete-orphan"
    )
```

`id: Mapped[int] = mapped_column(primary_key=True)` satırı `etkinlikler` tablosunda bir `id` sütunu (birincil anahtar) oluşturur; `baslik: Mapped[str]` ise bir `TEXT` sütunu oluşturur — tıpkı YG II'de elle yazdığınız aşağıdaki cümlenin karşılığı gibi, ama artık bu cümleyi elle yazmıyorsunuz:

```
CREATE TABLE etkinlikler (id INTEGER PRIMARY KEY, baslik TEXT, ...);
```

Sınıfın sonundaki `koltuklar` satırına şimdilik dikkat etmeyin — bu bir **ilişki** tanımıdır ve Ünite 2'nin konusudur.

Ortak Hata / Tuzak

Bu iki bildirimsel sınıf tanımını (Pydantic `BaseModel` ve SQLAlchemy `DeclarativeBase`) karıştırmak yaygın bir hatadır: ikisi de "sınıf tanımından bir şey türetir" görünse de, biri çalışma zamanında veri doğrular (Pydantic), diğeri bir veritabanı satırını temsil eder (SQLAlchemy). `EtkinlikDB`'ye `"kapasite": "abc"` gibi geçersiz bir değer atarsanız, Pydantic'in `Field(gt=0)` gibi bir kısıtı burada çalışmaz — ORM modeli veri doğrulamaz, yalnızca veritabanıyla konuşur. Doğrulama görevi HÂLÂ Pydantic

modellerinindir.

8.1.3 `engine`: veritabanına bağlanan tek nesne

motor (engine): Veritabanı bağlantısını ve SQL lehçesini yöneten, uygulama başına genelde tek nesne (*engine*).

Bir **motor (engine)**, hangi veritabanı dosyasına, hangi lehçeyle (SQLite, Postgres, ...) bağlanılacağını bilen, uygulama başına genelde tek bir nesnedir:

```
engine = create_engine("sqlite:///bilet.db", echo=False)
sqlite_yazma_kilidi_uygula(engine)
```

`"sqlite:///bilet.db"` dizgesi, proje kökünde `bilet.db` adlı bir SQLite dosyasına (dosya yoksa oluşturulur) bağlanmayı söyler. Hemen altındaki çağrının — bir yazma kilidi kuran yardımcı fonksiyon — ne yaptığını şimdilik merak etmeyin:

```
sqlite_yazma_kilidi_uygula(engine)
```

Bu, Ünite 4'ün eşzamanlı rezervasyon korumasının bir parçasıdır. `echo` parametresi `True` olduğunda her çalıştırılan SQL cümlesi konsola basılır; bunu Ünite 3'te aktif biçimde kullanacaksınız.

8.1.4 `Session`: bir çalışma birimi

oturum (session): Bir dizi sorgu/değişikliğin gruplandığı, commit/rollback ile sonlanan çalışma birimi (*session*).

`engine` veritabanına nasıl bağlanılacağını bilirken, fiilen sorgu/değişiklik yapan nesne **oturum (session)** adlı nesnedir. Aşağıdaki satır bir "oturum üretici" kurar, `SessionLocal` adında:

```
SessionLocal = sessionmaker(bind=engine)
```

Analoji

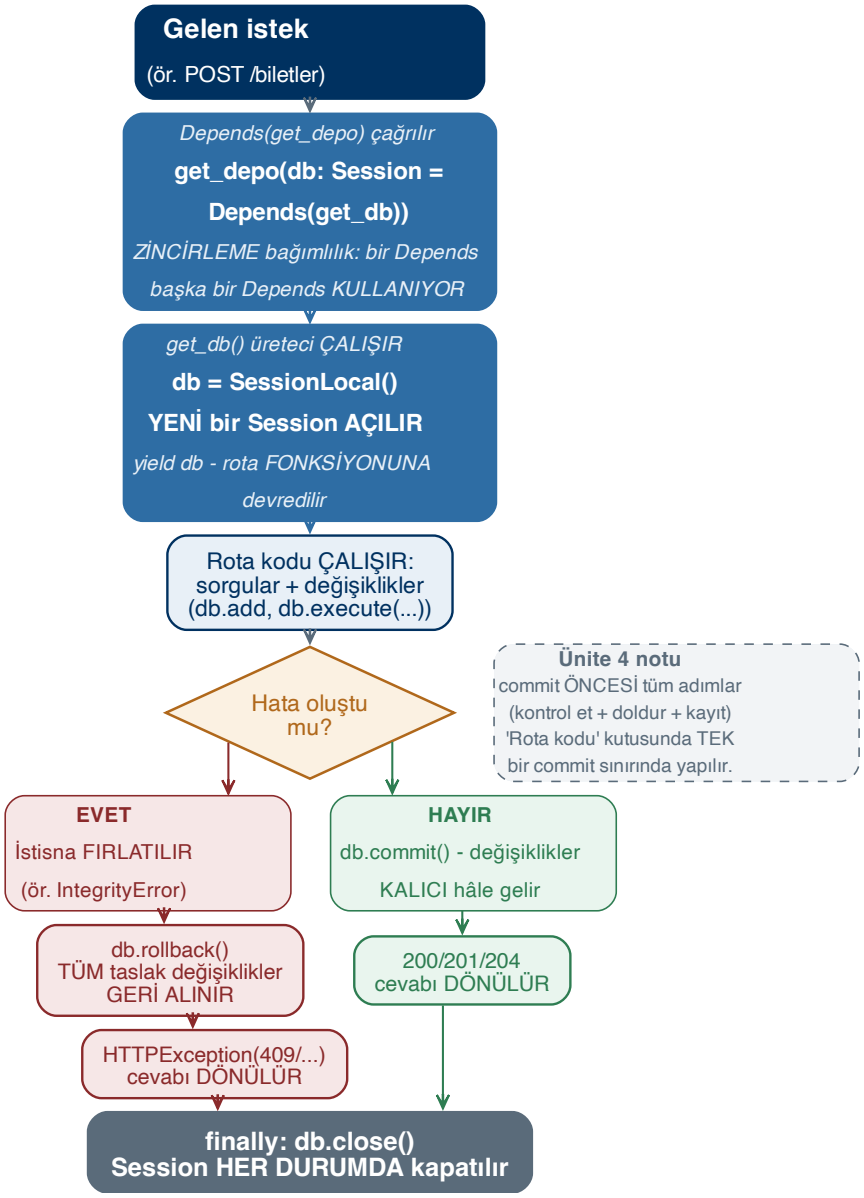
`engine`/`Session`'ı bir gişe defteri ve o deftere yazan memura benzetin: `engine` deftere (veritabanı dosyasına) erişebilen masadır; `Session` ise o masada oturup deftere yazan/okuyan memurdur — her "iş" (birkaç kayıt ekleme/güncelleme) memurun bir oturuşu süresince yapılır. Memur işini bitirince (`commit`) deftere kalıcı olarak yazar, yarıda bırakırsa (`rollback`) yazdıklarını siler.

`engine`'in aksine, `Session` uygulama başına tek bir nesne değildir — her istek için genelde yeni bir `Session` açılıp kapatılır. Bunu sağlayan üreteç (generator) bağımlılığı şöyledir:

```
def get_db() -> Generator[Session, None, None]:

    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`yield` İçeren bir FastAPI bağımlılığında, isteği işleyen kod `yield`'den önceki ve sonraki blokların arasına yerleşir: FastAPI önce `db` atama satırına kadar çalıştırır, sonra rota fonksiyonunuzu `db` değeriyle çağırır, rota fonksiyonu bittiğinde (hata olsa bile) `finally` bloğundaki `db.close()` çalışır. Bu, Hf.6 Ünite 4'ün "Hf.8'de `get_db_session()` bu yer tutucuyu değiştirecek" notunun gerçekleştiği yerdir. Şekil 8.1 bu akışı, `get_depo`→`get_db` zincirlemesinden başlayarak, başarı ve hata dallarıyla birlikte gösterir.



Şekil 8.1: Bir isteğin `get_db` üretecinden bir `Session` açılışına, sorgu/commit (başarı) ya da hata/rollback (hata) dallarına ve oturumun kesin kapanışına (`finally`) kadarki akışı.

8.1.5 Tablo oluşturma: `create_all`

Tanımladığınız sınıflar henüz veritabanında karşılık bulan tablolar değildir — bunları fiilen oluşturan çağrı `Base.metadata.create_all(engine)`'dir; sunucu `app/main.py` başlarken bir kez çağrılır:

```
Base.metadata.create_all(engine)
```

> **Dikkat**
>
> Bu çağrı mevcut bir tabloyu asla güncellemez: `EtkinlikDB`'ye yeni bir alan eklerseniz, `create_all` bunu varolan `etkinlikler` tablosuna yansıtmaz (tablo zaten var olduğu için dokunulmaz kalır). Gerçek bir projede şema değişikliklerini (yeni sütun, tip değişikliği) sürüm sürüm kaydeden bir araç kullanılır — bunun adı **şema göçü**dür (Alembic bu işi yapan popüler bir Python aracıdır). Bu ders Alembic'i kurmaz, yalnızca adını anar — `create_all`'ın boş bir arsaya ilk kez bina dikmesi ile Alembic'in sonradan tadilat kaydetmesi arasındaki fark, bir binanın ilk inşası ile zamanla yapılan genişletmeleri ayrı tutmaya benzer.

şema göçü: Veritabanı tablo yapısındaki değişiklikleri sürüm sürüm kaydeden işlem (*migration*).

8.1.6 ORM modeli vs Pydantic modeli: kritik ayrım

`EtkinlikDB` (`app/orm_modeller.py`) bir veritabanı satırının Python temsilidir — bir `Session`'a bağlıdır, tembel yükleme yapabilir, doğrudan JSON'a serileştirilmek için tasarlanmamıştır. `Etkinlik` (`app/modeller.py`, Hf.6'dan değişmeden kalır) ise dışa dönük JSON sözleşmesidir, veritabanından tamamen bağımsızdır. İki modeli tek bir sınıfta birleştirmek (ör. Pydantic modelini doğrudan veritabanı satırı yapmak) katmanları karıştırır — Hf.1 Ünite 3'ün "sunum/iş/veri katmanı" ayrımının bu haftaki somut karşılığı budur.

İki katman arasındaki köprü `model_validate(from_attributes=True)` çağrısıdır: bir `EtkinlikDB` nesnesinin öznitelik değerlerini okuyup bir `Etkinlik` Pydantic nesnesi üretir. `Depo` sınıfınızın `etkinlik_ekle` metodu bu köprüyü şöyle kullanır:

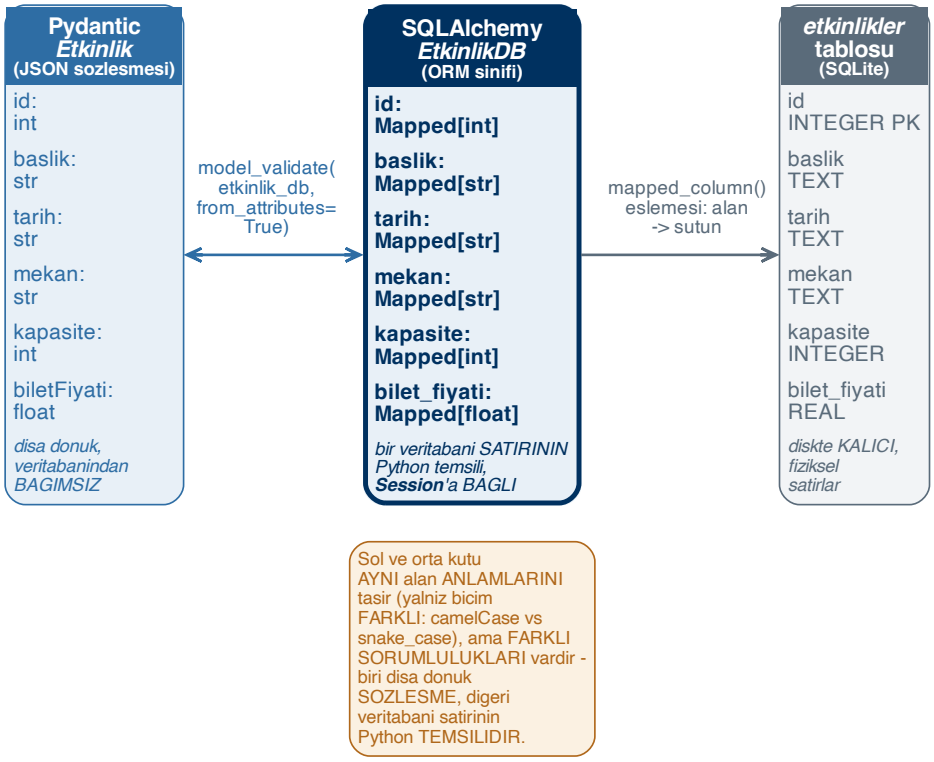
```
def etkinlik_ekle(self, veri: EtkinlikOlustur) -> Etkinlik:
    etkinlik_db = EtkinlikDB(**veri.model_dump())
    self.db.add(etkinlik_db)
    self.db.commit()
    self.db.refresh(etkinlik_db)
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)
```

Bu metodu izleyin: önce bir ORM nesnesi kurulur (`EtkinlikDB`), `add` ile `Session`'a eklenir, `commit` ile veritabanına kalıcı olarak yazılır, `refresh` ile veritabanının atadığı `id` gibi alanlar ORM nesnesine geri okunur — son satırda şu çağrı bu ORM nesnesini rota fonksiyonunun dönebileceği bir Pydantic nesnesine çevirir:

```
Etkinlik.model_validate(etkinlik_db, from_attributes=True)
```

Bu sayede rota fonksiyonlarının imzası (dönüş tipi hâlâ `Etkinlik`) Hf.6-7'den tek bir karakter değişmez — "iyi bir API sözleşmesi deponun nasıl çalıştığını gizler" dersinin somut kanıtı budur. Aynı `Depo` sınıfında `etkinlik_getir` ve `etkinlik_guncelle` metotları da aynı deseni (ORM nesnesini oku/güncelle, sonra `model_validate` ile Pydantic'e çevir) tekrarlar — tam kodları `kod_ornekleri/hafta08/sunucu/app/depo.py` dosyasındadır. Şekil 8.2 bu üç katmanı (Pydantic `Etkinlik`, ORM `EtkinlikDB`, SQL tablosu) yan yana gösterir.

```
def etkinlik_listele(
    self, sayfa: int = 1, boyut: int = 20, mekan: str | None = None
) -> list[Etkinlik]:
```



Şekil 8.2: Pydantic `Etkinlik` modeli, SQLAlchemy `EtkinlikDB` ORM sınıfı ve `etkinlikler` SQL tablosu; `model_validate( from_attributes=True)` köprüsü ORM nesnesinden Pydantic nesnesine akar.

```
    stmt = select(EtkinlikDB).order_by(EtkinlikDB.id)
    if mekan is not None:
        stmt = stmt.where(EtkinlikDB.mekan == mekan)
    stmt = stmt.limit(boyut).offset((sayfa - 1) * boyut)
    etkinlikler_db = self.db.execute(stmt).scalars().all()
    return [
        Etkinlik.model_validate(e, from_attributes=True) for e in
            etkinlikler_db
    ]
```

İpucu

`etkinlik_listele` metodundaki `select`/`order_by`/ `limit`/`offset` zincirine şimdiden göz atın — bunların ne anlama geldiğini Ünite 3'te tam olarak öğreneceksiniz; burada yalnızca bir `Session` üzerinden bir sorgu çalıştırmanın da aynı `db.execute(stmt)` deseniyle yapıldığını fark etmeniz yeterli.

3 • Katılım — Şimdi Siz Deneyin

`engine` ve `SessionLocal`'ı kurup `Base.metadata.create_all(engine)` çağırın; proje kökünde `bilet.db` dosyasının oluştuğunu (`ls` ile) doğrulayın. Ardından bir `EtkinlikDB` nesnesi oluşturup `db.add(...)` + `db.commit()` ile ekleyin, sonra `db.get(EtkinlikDB, 1)` ile geri okuyun.

Geribildirim. `bilet.db` dosyası proje kökünde belirir; `db.get` çağrısı eklediğiniz nesneyi (`id=1`) döndürür — Python nesneniz artık diskte kalıcı bir satırdır, sunucuyu kapatıp açsanız bile bu kayıt yerinde durur.

Kendinizi Sınayın 8.1

`EtkinlikDB` (ORM modeli) ile `Etkinlik` (Pydantic modeli) neden aynı sınıf değildir; `model_validate(from_attributes=True)` ne işe yarar?

5 • İzleme Aktarım — Başka Nerede?

Bugün kurduğunuz `engine`/`Session`/`get_db` deseni Hf.10'un `Kullanici` tablosunda, Hf.13'ün arka plan görevlerinde aynı şekilde kullanılacak; `DeclarativeBase` modeli yazma alışkanlığı dönem boyunca her yeni tabloda tekrarlanacak. Şimdi bu tek bir tablonun (`EtkinlikDB`) yalnız başına yaşamadığını — koltuklara ve biletlere nasıl bağlandığını — göreceksiniz.

8.2 İlişkiler (Etkinlik–Koltuk–Bilet)

1 • Ön-öğretim Neden Önemli?

Bir `EtkinlikDB` nesnesi oluşturdunuz, ama koltukları nasıl ekleyeceksiniz — her koltuğa "hangi etkinliğe ait" bilgisini elle bir sütuna mı yazacaksınız, yoksa `etkinlik.koltuklar` gibi doğal bir Python listesiyle mi çalışacaksınız? Ayrıca bir etkinlik silindiğinde koltukları/biletlerine ne olacak — Hf.6'da bu soru bilinçli olarak cevapsız bırakılmıştı; bu ünitede cevaplanır.

1 • Ön-öğretim Ön Bilgi

YG I/II'nin işaretçi/referansla bağlı veri yapılarını (bağlı liste/ağaç düğümleri) hatırlayın — bir düğümün başka bir düğüme bir işaretçiyle bağlanması, bu ünitenin `relationship`/`ForeignKey` kavramının kavramsal köprüsüdür. Hf.6 Ünite 3'ün "cascade silme bilinçli olarak yapılmaz, Hf.8'in konusudur" notunu da hatırlayın — bu ünitede bu karar kesinleşir.

Ünite 1'de tek başına bir `EtkinlikDB` nesnesi gördünüz. Ama Etkinlik/Bilet Sisteminizin gerçek gücü, üç tablonun birbirine nasıl bağlandığında ortaya çıkar: bir etkinliğin birden fazla koltuğu vardır, bir koltuğun en fazla bir aktif bileti olabilir. Bu ünite bu iki ilişkiyi nasıl kuracağınızı, SQLAlchemy'nin bu ilişkileri nasıl yüklediğini ve bir etkinlik silindiğinde bağlı kayıtlara ne olacağını gösterir.

8.2.1 Etkinlik 1–N Koltuk: yabancı anahtar

yabancı anahtar: Bir tablodaki bir sütunun başka bir tablonun birincil anahtarına işaret etmesi (*foreign key*).

Bir etkinliğin birden fazla koltuğu olabilir, ama her koltuk yalnızca bir etkinliğe aittir. Bu ilişkiyi kuran mekanizma, YG II'den hatırlayacağınız bir **yabancı anahtar**dır — `koltuklar` tablosundaki `etkinlik_id` sütunu, o koltuğun hangi etkinliğe ait olduğunu, o etkinliğin `id`'sini taşıyarak belirtir:

```
class KoltukDB(Base):

```

```
    __tablename__ = "koltuklar"

    id: Mapped[int] = mapped_column(primary_key=True)
    etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"))
    sira: Mapped[int]
    koltuk_no: Mapped[int]
    durum: Mapped[str] = mapped_column(default="bos")

    etkinlik: Mapped["EtkinlikDB"] = relationship(back_populates="koltuklar")
    biletler: Mapped[list["BiletDB"]] = relationship(
        back_populates="koltuk", cascade="all, delete-orphan"
    )
```

Az önce gördüğünüz kesitteki şu satır tam olarak bunu yapar:

```
etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"))
```

Ama bu tek başına size `koltuk.etkinlik` ya da `etkinlik.koltuklar` gibi rahat bir Python erişimi vermez — yalnızca veritabanı düzeyinde bir kısıt kurar. Python tarafındaki kolay erişimi sağlayan ayrı bir kavramdır: **ilişki (relationship)**. `KoltukDB` sınıfının sonundaki şu satır bunu kurar:

ilişki (relationship): ORM'de iki modelin yabancı anahtar üzerinden birbirine bağlı nesneler olarak erişilmesini sağlayan tanım (*relationship*).

```
etkinlik: Mapped["EtkinlikDB"] = relationship(back_populates="koltuklar")
```

Ünite 1'de gördüğünüz `EtkinlikDB` sınıfının `koltuklar` alanı da aynı ilişkinin karşı ucudur (`cascade="all, delete-orphan"` ile birlikte — birazdan bu haftanın cascade kararını açıklayacak).

`back_populates` parametresi, iki tanımın birbirini karşılıklı olarak tanımasını sağlar: `etkinlik.koltuklar` yazdığınızda SQLAlchemy arka planda bir sorgu çalıştırır, elle bir `JOIN/WHERE etkinlik_id = ?` yazmanıza gerek kalmaz.

Analoji

İlişki/yabancı anahtarı bir rezervasyon defterinde çapraz referansa benzetin: koltuk satırındaki bir sütun "hangi etkinliğe ait" diye o etkinliğin sıra numarasını taşır (yabancı anahtar); bilet satırı da "hangi koltuğa ait" diye o koltuğun numarasını taşır — defterin sayfaları böyle birbirine bağlanır.

8.2.2 Koltuk 1–0..1 Bilet: neden düz `UNIQUE` yetersiz

Bir koltuğun en fazla bir aktif bileti olabilir (hiç olmayabilir de — boş koltuk). Bu ilişki `BiletDB` tarafında aynı desenle kurulur:

```
class BiletDB(Base):

    __tablename__ = "biletler"

    id: Mapped[int] = mapped_column(primary_key=True)
    koltuk_id: Mapped[int] = mapped_column(ForeignKey("koltuklar.id"))
    kullanici_adi: Mapped[str]
    satin_alma_zamani: Mapped[str]
    durum: Mapped[str] = mapped_column(default="olusturuldu")

    koltuk: Mapped["KoltukDB"] = relationship(back_populates="biletler")
```

Sınıfın docstring'inde önemli bir not var: iptal edilmiş biletler Hf.6'daki gibi silinmez, `durum='iptal'` olarak saklanır (denetim amaçlı). Bu, bir koltuğun zaman içinde birden fazla bilet kaydına sahip olabileceği (biri aktif, gerisi iptal) anlamına gelir — bu yüzden `koltuk_id` üzerinde düz bir `UNIQUE` kısıtı yetersizdir (o kısıt

ikinci, iptal edilmiş bir bilet kaydını bile reddederdi). Bu sorunun çözümü — kısmi/koşullu bir `UNIQUE` indeks — Ünite 4'ün konusudur.

8.2.3 Tembel (lazy) vs istekli (`selectinload`) yükleme

`etkinlik.koltuklar` yazdığınızda, varsayılan davranış (`lazy="select"`) o ilişkiye ilk erişildiğinde ayrı bir SQL sorgusu tetikler. Bir listedeki her etkinlik için ayrı ayrı bu erişim yapıldığında **N+1 sorunu** oluşur: 1 ana sorgu (etkinlikleri getirmek için) + N ilişki sorgusu (her etkinliğin koltuklarını ayrı ayrı getirmek için). 100 etkinlik listelediğinizde bu 101 SQL sorgusu demektir.

N+1 sorunu: Bir listedeki her kayıt için ayrı bir ilişki sorgusu çalıştırılması yüzünden sorgu sayısının patlaması (*N+1 problem*).

Analoji

Tembel/istekli yüklemeyi bir öğretmenin sınıf listesini almasına benzetin: önce sınıf listesini görüp her öğrencinin notunu ayrı ayrı dosyadan çekmek (**tembel yükleme** — her öğrenci için ayrı bir gidiş) ile hepsinin notlarını tek seferde getiren bir rapor istemek (**istekli yükleme**, *eager loading*: ilişkili kayıtların ana sorguyla birlikte, önceden ve toplu çekilmesi) arasındaki fark — birincisi öğretmeni dosya dolabına N kere, ikincisi tek kere yürütür.

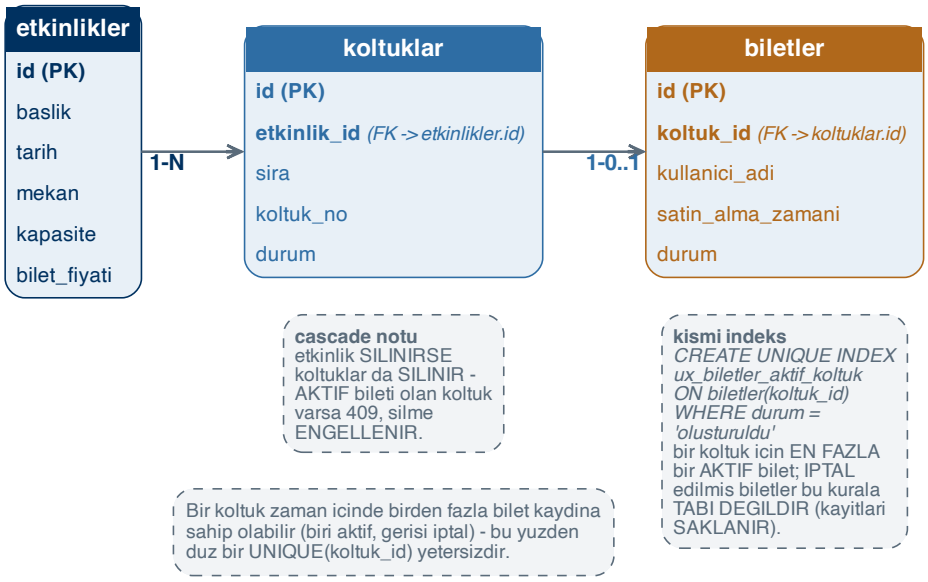
tembel yükleme: İlişkili kayıtların yalnızca erişildiği anda ayrı bir sorguyla çekilmesi (*lazy loading*).

İstekli yükleme, ilişkiyi önceden, tek bir ek sorguda çeker:

```
from sqlalchemy.orm import selectinload

stmt = select(EtkinlikDB).options(selectinload(EtkinlikDB.koltuklar))
etkinlikler = db.execute(stmt).scalars().all()
```

Bu tek satırlık `.options(selectinload(...))` eki, sorgu sayısını etkinlik sayısından bağımsız, sabit tutar (1 ana sorgu + 1 istekli sorgu). Şekil 8.3 bu iki tablo arasındaki ilişkiyi, yabancı anahtarları ve Ünite 4'ün kısmi `UNIQUE` indeksini bir arada gösterir.



Şekil 8.3: Etkinlik 1–N Koltuk 1–0..1 Bilet ilişkisi, yabancı anahtarlar ve aktif bir bilet için tanımlı kısmi `UNIQUE(koltuk_id)` indeksi.

> **İpucu**
>
> N+1 sorununun performans derinliği (kaç sorgu, ne kadar yavaşlama, hangi durumda gerçekten önemli) bu haftanın kapsamı dışındadır — bunu Hf.15'in performans ünitesinde derinlemesine göreceksiniz. Burada yalnızca sorunun varlığını ve çözümünün adını (`selectinload`) tanımanız yeterlidir.

8.2.4 Cascade silme kararı: aktif bilet varsa `409`, yoksa kademeli

Hf.6'da bilinçli olarak ertelenen soru şuydu: bir etkinlik silindiğinde bağlı koltuklara/biletlere ne olur? Bu hafta kararı kesinleşir: `DELETE /etkinlikler/{id}` çağrıldığında önce o etkinliğe bağlı koltuklardan herhangi birinin aktif bir bileti (`durum='olusturuldu'`) olup olmadığı kontrol edilir:

```
def etkinlik_sil(self, etkinlik_id: int) -> str:

    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return "bulunamadi"

    aktif_bilet_var = any(
        bilet.durum == "olusturuldu"
        for koltuk in etkinlik_db.koltuklar
        for bilet in koltuk.biletler
    )
    if aktif_bilet_var:
        return "aktif_bilet_var"

    self.db.delete(etkinlik_db)
    self.db.commit()
    return "silindi"
```

Aktif bilet varsa istek `409 Conflict` ile reddedilir — "bu etkinliğin satılmış biletleri var, önce iptal edilmelidir" benzeri bir mesajla. Yoksa (tüm koltuklar boş ya da yalnızca iptal edilmiş biletleri varsa) etkinlik silinir; Ünite 1'de gördüğünüz `cascade="all, delete-orphan"` ayarı sayesinde bağlı tüm koltuklar (ve onların cascade'iyle varsa iptal edilmiş biletleri) otomatik olarak kademeli silinir — `self.db.delete(etkinlik_db)` çağrısının tek başına yaptığı bu.

> **Analoji**
>
> Cascade silmeyi bir etkinliğin iptaline benzetin: bir konser tamamen iptal edildiğinde, o konserin boş koltukları da anlamsız hâle gelir ve birlikte silinir — ama bileti satılmış bir koltuk varsa, önce o biletlerin ne olacağına karar vermeden konseri silemezsiniz.

Bu kademeli silme davranışı rota katmanına hiç sızmaz — rota fonksiyonu yalnızca `Depo`'nun döndürdüğü sonucu (`"silindi"` / `"bulunamadi"` / `"aktif_bilet_var"`) durum koduna çevirir:

```
@router.delete("/{etkinlik_id}", status_code=204)
def etkinlik_sil(etkinlik_id: int, depo: Depo = Depends(get_depo)) -> None:
    sonuc = depo.etkinlik_sil(etkinlik_id)
    if sonuc == "bulunamadi":
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    if sonuc == "aktif_bilet_var":
```

```
        raise HTTPException(
            status_code=409,
            detail="Bu etkinliğin satılmış biletleri var, önce iptal edilmelidir
                ",
        )
```

Ortak Hata / Tuzak

`cascade="all, delete-orphan"` yalnızca `relationship` tanımlanmış nesneler arasında çalışır — veritabanındaki ham bir `FOREIGN KEY` kısıtı tek başına kademeli silme yapmaz (SQLite varsayılan olarak yabancı anahtar kısıtlarını bile uygulamaz, yalnızca SQLAlchemy'nin ORM düzeyindeki `cascade` ayarı bu davranışı sağlar). Bir `KoltukDB` nesnesini `relationship` tanımı olmadan, yalnızca `etkinlik_id` sütununu elle sıfırlayarak "koparmaya" çalışmak, cascade'in beklediğiniz gibi çalışmasını engeller.

3 • Katılım — Şimdi Siz Deneyin

`KoltukDB`'ye `ForeignKey('etkinlikler.id')` ile bir `etkinlik_id` sütunu ekleyin; `EtkinlikDB`'ye `relationship(back_populates='etkinlik')` ile `koltuklar` alanını tanımlayın. Bir etkinliğe 3 koltuk ekleyip `etkinlik.koltuklar` ile geri okuyun.

Geribildirim. `etkinlik.koltuklar` bir Python listesi gibi davranır, ama arka planda SQLAlchemy sizin yerinize bir `SELECT` çalıştırır — elle bir JOIN/`WHERE etkinlik_id = ?` yazmanıza gerek kalmaz.

Kendinizi Sınayın 8.2

`GET /etkinlikler` isteğinde her etkinliğin koltuk sayısını ayrı ayrı bir sorguyla çekerseniz (tembel yükleme, N etkinlik için N ek sorgu) nasıl bir sorun oluşur, nasıl önlenir?

5 • İzleme — Aktarım — Başka Nerede?

Etkinlik-Koltuk-Bilet ilişki zinciri Hf.10'da `Kullanici`-Bilet ilişkisiyle (bir kullanıcının biletleri) genişleyecek; `selectinload` deseni Hf.15'in performans ünitesinde derinleşecek. Şimdi bu ilişkileri sorgularken SQLAlchemy'nin arkada gerçekte ne yaptığını — ve bunu YG II'nin elle yazdığınız SQL'iyle nasıl karşılaştıracağınızı — göreceksiniz.

8.3 Sorgular, Sayfalama ve Ham SQL Karşılığı

1 • Ön-öğretim — Neden Önemli?

Deponuz artık tüm kayıtları listeliyor, ama 500 etkinlik olsa tek seferde hepsini döndürmek hem sunucuyu hem tarayıcıyı yorar; ayrıca ORM'in arkada ne SQL ürettiğini görmeden güvenmek, YG II'de öğrendiğiniz "ne olduğunu bil" disiplinine aykırıdır. Bu ünite her iki sorunu da çözüyor.

1 • Ön-öğretim **Ön Bilgi**

YG II'nin `SELECT/WHERE/ORDER BY/LIMIT` temel SQL söz dizimini hatırlayın. Hf.6 Ünite 1'in sorgu parametresi kavramını (`?mekan=...`) da hatırlayın — bu ünitenin `?sayfa=&boyut=` sayfalama parametreleri bunun doğrudan devamıdır.

Ünite 1'de `depo_etkinlik_metotlari` kesitinde `etkinlik_listele` metodunu gördünüz ama üzerinde durmadınız. Bu ünite tam olarak o metodun içindeki sorgu satırlarına odaklanır: SQLAlchemy'nin `select()` API'sini, sayfalamayı, ORM'in ürettiği gerçek SQL'i görmeyi ve ORM'in yetersiz kaldığı durumlarda kaçış kapısını.

8.3.1 `select()` API'si: YG II'nin `SELECT` cümlesinin Python nesnesi

SQLAlchemy 2.x'te bir sorgu, doğrudan çalıştırılan bir dizge değil, bir Python nesnesi olarak inşa edilir:

```
stmt = select(EtkinlikDB).where(EtkinlikDB.mekan == mekan).order_by(
    EtkinlikDB.id)
sonuc = db.execute(stmt).scalars().all()
```

Bu iki satır Ünite 1'de gördüğünüz `etkinlik_listele` metodundan birebir alınmıştır. `select(EtkinlikDB)` YG II'nin `SELECT * FROM etkinlikler` cümlesinin karşılığıdır; `.where(...)` bir `WHERE` koşulu, `.order_by(...)` bir `ORDER BY` koşulu ekler — bu metotlar zincirlenebilir, her biri yeni bir `stmt` nesnesi döndürür. `db.execute(stmt)` sorguyu fiilen çalıştırır; `.scalars()` dönen satırlardan doğrudan `EtkinlikDB` nesnelerini çıkarır (aksi hâlde satır demetleri alırdınız); `.all()` hepsini bir listeye toplar.

8.3.2 `limit/offset` ile sayfalama

Hf.6 Ünite 1'in `?mekan=...` sorgu parametresi kavramının doğrudan devamı olarak, mevcut liste uç noktalarına isteğe bağlı iki yeni sorgu parametresi eklenir: `sayfa` (varsayılan 1) ve `boyut` (varsayılan 20). Rota tarafında değişen tek şey budur:

```
@router.get("", response_model=list[Etkinlik])
def etkinlik_liste(
    mekan: str | None = None,
    sayfa: int = 1,
    boyut: int = 20,
    depo: Depo = Depends(get_depo),
) -> list[Etkinlik]:
    return depo.etkinlik_listele(sayfa=sayfa, boyut=boyut, mekan=mekan)
```

sayfalama: Büyük bir sonuç kümesini küçük parçalar hâlinde döndürme (*pagination*).

Dönüş tipi hâlâ bir `list[Etkinlik]`'tir — bir **sayfalama** sarmalayıcı nesnesi (ör. `{"toplam": ..., "sonuclar": [...]}`) bu hafta kullanılmaz; OpenAPI şeması bu yüzden geriye uyumlu kalır — Hf.7'nin `istemci/` kodu tek bir satır değişmeden çalışmaya devam eder. `Depo`'daki karşılığı şudur:

```
stmt = stmt.limit(boyut).offset((sayfa - 1) * boyut)
```

Analoji

Sayfalamayı bir telefon rehberinin ciltlerine benzetin: binlerce kişiyi tek ciltte basmak yerine, "A-D", "E-H" gibi ciltlere (sayfalara) bölersiniz — her seferinde

istediğiniz cildi (sayfa numarasını) istersiniz, tüm rehberi taşımazsınız.

Gerçek bir istek şöyle çalışır — sunucu çalışırken `sayfa=1`, `boyut=2` ile en fazla iki kayıt döner:

```
curl -i "$BASE_URL/etkinlikler?sayfa=1&boyut=2"
```

`pytest` testinde bu davranış iki farklı sayfada ve varsayılan (parametresiz) çağrıda doğrulanır. Kurulum bölümü önce üç etkinlik oluşturur:

```
def test_etkinlik_sayfalama(client):
   for sira in range(3):
      client.post(
         "/etkinlikler",
         json={
            "baslik": f"Etkinlik {sira}",
            "tarih": "2026-11-14",
            "mekan": "Salon A",
            "kapasite": 100,
            "biletFiyati": 250,
         },
      )
```

Ardından aynı testin doğrulama bölümü, `boyut=2` ile ilk sayfada iki kayıt, ikinci sayfada kalan tek kaydı, parametresiz çağrıda ise varsayılan `sayfa=1`/`boyut=20` ile tüm üç kaydı bekler:

```
ilk_sayfa = client.get("/etkinlikler", params={"sayfa": 1, "boyut": 2})
assert ilk_sayfa.status_code == 200
assert len(ilk_sayfa.json()) == 2

ikinci_sayfa = client.get("/etkinlikler", params={"sayfa": 2, "boyut": 2})
assert ikinci_sayfa.status_code == 200
assert len(ikinci_sayfa.json()) == 1

varsayilan_sayfalama = client.get("/etkinlikler")
assert len(varsayilan_sayfalama.json()) == 3
```

İpucu

Toplam kayıt sayısını (sayfalama dışı, tüm eşleşen kayıt sayısı) hesaplamak isterseniz `func.count` kullanılır (`select(func.count()).select_from(EtkinlikDB)`); bunu bir `X-Toplam-Kayit` yanıt başlığı olarak döndürmek yaygın bir desendir. Bu hafta zorunlu bir kod örneği içermez — istemci tarafında "kaç sayfa var" göstermek istediğinizde ileride ihtiyaç duyacağınız bir araçtır.

8.3.3 Join: koltukları etkinlikle birlikte çekmek

Ünite 2'nin `selectinload` örneğine ek olarak, bir etkinliği koltuklarıyla birlikte tek bir sorguda almanın klasik SQL karşılığı bir `JOIN` cümlesidir. `sorgular.sql` dosyasındaki elle yazılmış karşılığı şöyledir:

```
SELECT etkinlikler.id, etkinlikler.baslik, koltuklar.id AS koltuk_id,
     koltuklar.sira, koltuklar.koltuk_no, koltuklar.durum
FROM etkinlikler
JOIN koltuklar ON koltuklar.etkinlik_id = etkinlikler.id
WHERE etkinlikler.id = 1;
```

Bu, `selectinload(EtkinlikDB.koltuklar)`'ın çözdüğü aynı sorunun (ilişkili verileri tek sorguda almak) elle yazılmış bir yoludur — `selectinload` arka planda tam olarak böyle bir sorgu çalıştırmaz (kendi stratejisini kullanır), ama amaç aynıdır.

8.3.4 `echo=True` ile ORM'in ürettiği SQL'i görmek

`create_engine(..., echo=True)` Açıkken, her `db.execute(...)` çağrısının arkasında çalıştırdığı gerçek SQL cümlesi (parametreleriyle birlikte) konsola basılır. Aşağıdaki sorguyu `echo=True` ile çalıştırdığınızda:

```
select(EtkinlikDB).where(EtkinlikDB.mekan == "Salon B").order_by(EtkinlikDB.
    id)
```

konsolda şuna benzer bir çıktı görürsünüz:

Beklenen Çıktı

```
INFO sqlalchemy.engine.Engine SELECT etkinlikler.id, etkinlikler.baslik,
etkinlikler.tarih, etkinlikler.mekan, etkinlikler.kapasite,
etkinlikler.bilet_fiyati
FROM etkinlikler
WHERE etkinlikler.mekan = ? ORDER BY etkinlikler.id
INFO sqlalchemy.engine.Engine [generated in 0.00010s] ('Salon B',)
```

`sorgular.sql` dosyasındaki elle yazılmış karşılığını yan yana koyun:

```
SELECT etkinlikler.id, etkinlikler.baslik, etkinlikler.tarih,
     etkinlikler.mekan, etkinlikler.kapasite, etkinlikler.bilet_fiyati
FROM etkinlikler
WHERE etkinlikler.mekan = 'Salon A'
ORDER BY etkinlikler.id;
```

İkisi aynı sonucu üretir, ama biçim farklıdır: ORM sorguyu kendi iç temsilinden (Python nesne modelinden) üretir, insan gibi "en kısa/en okunaklı" SQL yazmaya çalışmaz — sütun adları tam tablo adıyla öneklenmiş (`etkinlikler.id`), değer bir `?` yer tutucusuyla ayrı gönderilmiş (parametreli sorgu, SQL enjeksiyonuna karşı güvenli bir yan etki). Bu bir sorun değildir; fonksiyonel olarak aynı sonucu verir.

Aynı sayfalama sorgusunun elle yazılmış SQL karşılığı da `sorgular.sql` dosyasında durur:

```
SELECT etkinlikler.id, etkinlikler.baslik, etkinlikler.tarih,
     etkinlikler.mekan, etkinlikler.kapasite, etkinlikler.bilet_fiyati
FROM etkinlikler
ORDER BY etkinlikler.id
LIMIT 10 OFFSET 10;
```

8.3.5 `text()`: ORM'in yetmediği yerde kaçış kapısı

ORM'in `select()` API'si rahat ifade edemediği karmaşık bir rapor ya da performans-kritik bir sorgu gerektiğinde, `db.execute(text("SELECT ..."))` ile ham bir SQL cümlesi çalıştırabilirsiniz:

```
sonuc = db.execute(text("SELECT COUNT(*) FROM biletler WHERE durum = :d"), {"
    d": "olusturuldu"})
```

Genel kural şudur: "`select()` API'si yeterliyse onu kullanın, yetmiyorsa `text()`'e düşün." Bu hafta zorunlu bir kullanım örneği içermez — `text` fonksiyonunun adı `sorgular.sql` dosyasının başlığında gösterilir, siz Ünite 4'e kadar bu kaçış kapısına hiç ihtiyaç duymayacaksınız.

Karmaşıklık (big-O)

ORM'in ürettiği SQL, elle yazdığınızdan bazen daha fazla sütun seçebilir (ör. `SELECT *` yerine tüm sütunları tek tek adlandırma) ya da farklı bir JOIN stratejisi kullanabilir — bu genelde performans açısından önemsizdir, ama büyük tablolarda `echo=True` ile üretilen SQL'i periyodik olarak gözden geçirmek iyi bir alışkanlıktır.

3 • Katılım Şimdi Siz Deneyin

`echo=True` açıp aşağıdaki sorguyu çalıştırın:

```
select(EtkinlikDB).where(EtkinlikDB.mekan == 'Salon')
```

Konsolda basılan SQL'i okuyup `sorgular.sql`'deki elle yazılmış karşılığıyla karşılaştırın.

Geribildirim. Konsolda `WHERE etkinlikler.mekan = ?` benzeri bir satır ve parametre değeri görünür — ORM'in ürettiği SQL, yapısal olarak elle yazdığınızla aynı mantığı taşır, yalnızca ORM bunu sizin yerinize, Python nesnelerinden üretir.

Kendinizi Sınayın 8.3

`echo=True` açıkken ORM'in ürettiği SQL, YG II'de elle yazdığınız SQL'den neden farklı görünebilir (ör. takma adlar, sütun sırası) — bu bir sorun mudur?

5 • İzleme Aktarım — Başka Nerede?

`echo=True` ile SQL okuma alışkanlığı Hf.15'in performans ünitesinde "hangi sorgu yavaş" teşhisinde aynen kullanılacak; `text()` ile ham SQL yazma becerisi ORM'in ifade edemediği karmaşık raporlarda dönem boyunca elde tutulacak bir yedek araçtır. Şimdi bu sorguların güvendiğiniz tek koltuğa iki kişinin aynı anda başvurmasına karşı nasıl korunduğunu göreceksiniz.

8.4 Transaction ve Eşzamanlı Rezervasyon

1 • Ön-öğretim Neden Önemli?

Bugüne kadar "aynı koltuğu iki kullanıcı aynı anda alırsa" sorusu hiç gerçek bir tehdit değildi — Hf.6'nın bellekteki `dict`'i tek bir Python sürecinde, istekler pratikte sırayla işleniyordu. Şimdi verileriniz bir dosyada, ve gerçek bir üretim sunucusunda birden fazla istek aynı anda işlenmeye çalışabilir — bu haftanın son ve en önemli sorusu tam olarak budur.

1 • Ön-öğretim Ön Bilgi

YG II'nin ACID/transaction kavramını (SQLite dersinde görülmüş) hatırlayın. Ünite 2/3'ün ORM sorgu ve ilişki bilgisi de gereklidir — bu ünitede transaction içinde hangi sorguların çalıştığını göreceksiniz.

Ünite 3'ün sonunda sorguları güvenle okuyabiliyorsunuz. Ama bir `POST /biletler` isteği yalnızca bir sorgu değildir — üç adımlık bir işlemdir: koltuğu kontrol et, dolu yap, bileti yaz. Bu ünite bu üç adımın nasıl tek bir bütün olarak korunduğunu, YG II'nin ACID kavramının burada nasıl somutlaştığını gösterir.

8.4.1 ACID hatırlatma: bu haftanın odağı atomiklik ve yalıtım

işlem (transaction): Bir dizi veritabanı işleminin ya tamamen uygulandığı ya da hiç uygulanmadığı bütünlük birimi (*transaction*).

YG II'de **işlem (transaction)** kavramının dört özelliğini (ACID) gördünüz: Atomiklik (ya hep ya hiç), Tutarlılık, Yalıtım ve Kalıcılık. Bu haftanın ana odağı atomiklik ve yalıtımdır — diğer ikisi (tutarlılık, kalıcılık) veritabanı motorunun kendisinin garantisidir, bu haftada uygulama kodu seviyesinde değiştirilmez.

8.4.2 `Session` içinde `commit/rollback`

Bir `Session`'a yapılan `db.add(...)`/değişiklikler veritabanına hemen yazılmaz — `db.commit()` çağrılana kadar bir "taslak" hâlinde bekler; bir hata oluşursa `db.rollback()` ile tüm taslak değişiklikler geri alınır, veritabanı işlem başlamadan önceki hâline döner. Bunu Ünite 1'de `etkinlik_ekle` metodunda zaten gördünüz (`self.db.commit()`); bu ünite aynı mekanizmayı, hata durumunda `rollback` çağırarak genişletir.

8.4.3 Neden bu sorun Şimdi görünür oldu

Hf.6'nın bellekteki `dict`'inde bu sorun hiç gözlemlenemedi — tek bir Python süreci, gelen istekleri (varsayılan `uvicorn` çalıştırmasında) pratikte sırayla işliyordu; "aynı anda" iki isteğin gerçekten çakışması neredeyse İMKÂNSIZDI. Bu hafta veri bir dosyada (disk G/Ç'si, ayrı bağlantılar) yaşadığından ve gerçek bir üretim ortamında birden fazla istek/bağlantı gerçekten eşzamanlı ilerleyebileceğinden, bu **yarış durumu** artık gerçek bir tehdittir.

yarış durumu: İki eşzamanlı işlemin aynı veriye çakışan biçimde erişip beklenmeyen bir sonuç üretmesi (*race condition*).

> **Analoji**
>
> Yarış durumunu aynı koltuğa aynı anda iki gişeden başvuruya benzetin: iki farklı gişe memuru aynı anda "12. koltuk boş mu?" diye deftere bakarsa, her ikisi de "boş" görüp aynı koltuğu iki kişiye satabilir — deftere yazarken birinin diğerini beklemesi (kilit) bu çakışmayı önler.

8.4.4 Çözüm: kontrol et + dolu yap + yaz — Tek transaction

`POST /biletler` işleyicisinin içindeki üç adım — (a) koltuğun var ve `'bos'` olduğunu kontrol etme, (b) koltuğun `durum`'unu `'dolu'`ya çevirme, (c) yeni bir `BiletDB` satırı ekleme — tek bir `Session` ömründe, tek bir `db.commit()` sınırında yapılmalıdır. Adım (a), koltuğu okuyup iki olası hatayı (koltuk yok, koltuk zaten dolu) erken keser:

```
koltuk_db = self.db.get(KoltukDB, veri.koltuk_id)
if koltuk_db is None:
    raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
if koltuk_db.durum == "dolu":
    raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")
```

Adım (b) ve (c) koltuğu `'dolu'` işaretler ve yeni bir `BiletDB` nesnesini `Session`'a ekler — bu iki değişiklik henüz veritabanına yazılmamıştır, yalnızca aynı `Session`'ın taslağındadır:

```
koltuk_db.durum = "dolu"
bilet_db = BiletDB(
    koltuk_id=veri.koltuk_id,
    kullanici_adi=veri.kullanici_adi,
    satin_alma_zamani=datetime.now().isoformat(),
    durum="olusturuldu",
)
self.db.add(bilet_db)
```

Analoji

Bu "tek kalem kuralı"nı bir gişe memurunun bir rezervasyonu tek kalemde yazmasına benzetin: "koltuğu kontrol et, boşsa dolu işaretle, bileti yaz" — bu üç adım tek bir kalem darbesi gibi ya hep ya hiç olmalı; memur ikinci adımda kalemi bırakıp araya başka bir iş sokarsa (yarıda kesilen işlem), defter yarım güncellenmiş, tutarsız bir hâlde kalır.

Adımlar ayrı ayrı `commit` edilseydi (kontrol et → COMMIT, güncelle → COMMIT), iki commit arasındaki boşlukta başka bir isteğin araya girip aynı koltuğu alması mümkün olurdu — tek transaction bu boşluğu tamamen kapatır.

8.4.5 SQLite yazma kilidi: `BEGIN IMMEDIATE`

Ünite 1'de `sqlite_yazma_kilidi_uygula(engine)` çağrısını görüp ertelemiştiniz — şimdi sırası geldi. SQLite'ın varsayılan transaction'ı (`BEGIN DEFERRED`) bir yazma işlemine kadar kilit almaz; `BEGIN IMMEDIATE` ile transaction başlar başlamaz bir yazma kilidi (RESERVED) alınır — aynı anda başka bir transaction'ın yazma niyetiyle başlaması engellenir. İkinci istek ya kısa bir süre bekler ya da "database is locked" hatası alır. Bunun elle SQL karşılığı şöyledir:

```
BEGIN IMMEDIATE;

SELECT id, durum FROM koltuklar WHERE id = 1;

UPDATE koltuklar SET durum = 'dolu' WHERE id = 1;

INSERT INTO biletler (koltuk_id, kullanici_adi, satin_alma_zamani, durum)
VALUES (1, 'Ayşe Yılmaz', '2026-11-14T10:00:00', 'olusturuldu');

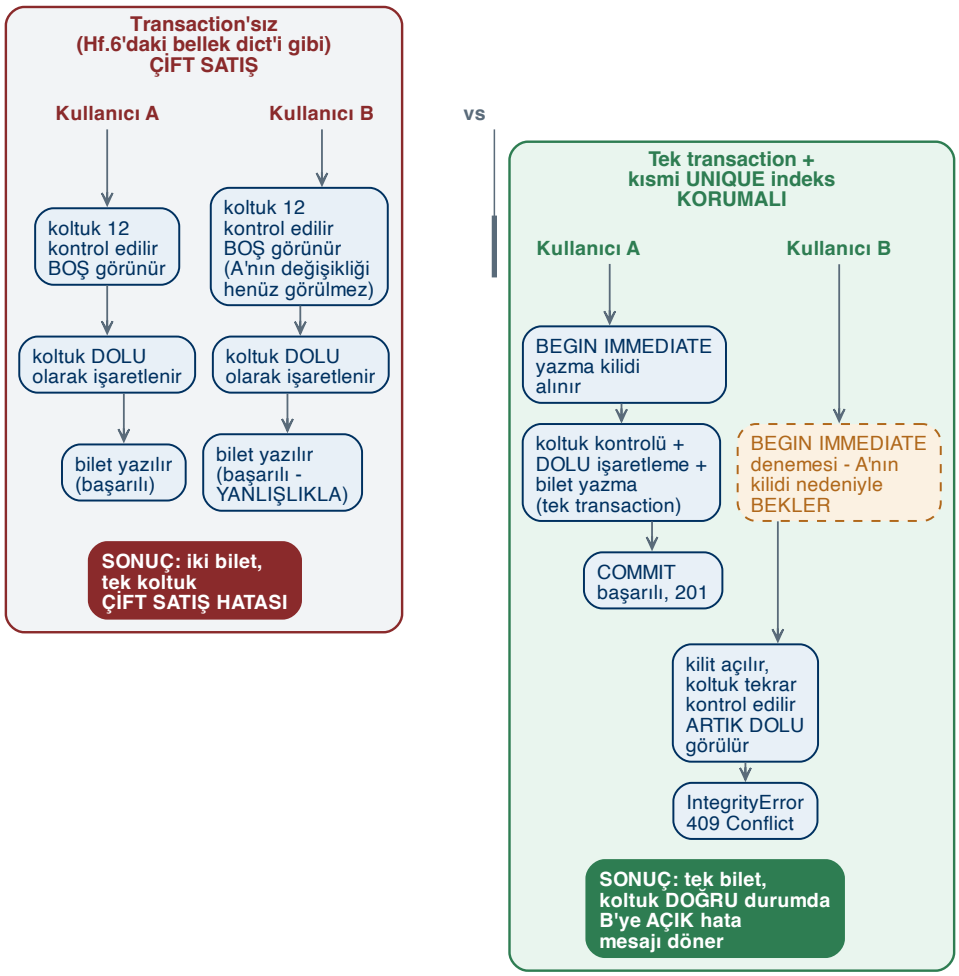
COMMIT;
```

Bu üç adımın (kontrol et, güncelle, yaz) `BEGIN IMMEDIATE` ile `COMMIT` arasında, tek bir yazma kilidi altında yürüdüğünü fark edin — aynı koltuğa eşzamanlı gelen ikinci istek, birincisi commit edene kadar tam burada bekler ve koltuğun artık dolu olduğunu görür. Şekil 8.4 bu korumanın olduğu ve olmadığı iki senaryoyu yan yana gösterir.

8.4.6 İkinci savunma katmanı: kısmi (koşullu) `UNIQUE` indeks

Uygulama kontrolü (`if koltuk_db.durum == "dolu"`) herhangi bir nedenle atlanırsa (ör. ilerideki bir kod değişikliği bu kontrolü yanlışlıkla silerse), son bir güvence katmanı olarak Ünite 2'de sözü edilen kısmi `UNIQUE` indeks devreye girer:

```
Index(
    "ux_biletler_aktif_koltuk",
    BiletDB.koltuk_id,
```



Şekil 8.4: İki eşzamanlı rezervasyon isteğinin transaction'sız (çift satış hatası) ve tek transaction/`UNIQUE` indeksli (ikincinin 409 alması) akışlarının karşılaştırması.

```
    unique=True,
    sqlite_where=(BiletDB.durum == "olusturuldu"),
)
```

Bu indeksin ürettiği ham DDL (Data Definition Language) cümlesi şudur:

```
CREATE UNIQUE INDEX ux_biletler_aktif_koltuk
ON biletler (koltuk_id)
WHERE durum = 'olusturuldu';
```

Bir koltuk için en fazla bir aktif (`durum='olusturuldu'`) bilet kaydına izin veren bu indeks, veritabanının kendisinin uyguladığı bir kuraldır. Uygulama kontrolü bu kısıtı ihlal ederse, SQLAlchemy bir **bütünlük hatası** (`IntegrityError`) fırlatır; `bilet_ekle` metodunun son adımı tam olarak bu istisnayı yakalayıp `409`'a çevirir:

bütünlük hatası: Bir veritabanı kısıtının (UNIQUE, foreıgn key vb.) ihlal edildiğinde fırlatılan hata (*integrity error*).

```
try:
    self.db.commit()
except IntegrityError:
    self.db.rollback()
    raise HTTPException(
        status_code=409, detail="Bu koltuk zaten satılmış"
    ) from None
self.db.refresh(bilet_db)
return Bilet.model_validate(bilet_db, from_attributes=True)
```

Analoji

`UNIQUE` kısıtını defterin kendisinin bir kural dayatmasına benzetin: memur dikkatsiz davransa bile, defterin kendisi ("bu koltuk numarası için sadece bir aktif bilet kaydı olabilir") kuralını uygular ve ikinci kaydı reddeder — bu, memurun (uygulama kodunun) olası bir hatasına karşı son bir güvence katmanıdır.

Gerçek bir sunucuda bu davranış şöyle görünür — ilk istek başarılı, ikincisi aynı koltuk için `409` alır:

Beklenen Çıktı

```
$ curl -i -X POST http://127.0.0.1:8000/biletler \
    -H "Content-Type: application/json" \
    -d '{"koltukId": 1, "kullaniciAdi": "Ayşe Yılmaz"}'
HTTP/1.1 201 Created
content-type: application/json

{"koltukId":1,"kullaniciAdi":"Ayşe Yılmaz","id":1,
 "satinAlmaZamani":"2026-09-07T15:41:17.611828","durum":"olusturuldu"}

$ curl -i -X POST http://127.0.0.1:8000/biletler \
    -H "Content-Type: application/json" \
    -d '{"koltukId": 1, "kullaniciAdi": "Mehmet Demir"}'
HTTP/1.1 409 Conflict
content-type: application/json

{"detail":"Bu koltuk zaten satılmış"}
```

İpucu

Gerçek bir üretim sisteminde "database is locked" hatası alan bir isteğin, küçük bir gecikmeyle (ör. birkaç milisaniye) yeniden denenmesi (retry) yaygın bir de-

sendir — bu ders bu deseni kod olarak yazmaz, yalnızca fikrini anar: geçici kilit çakışmaları çoğu zaman kısa bir bekleme sonrası kendiliğinden çözülür.

8.4.7 `pytest` ile transaction testi

Bu korumayı gözle değil, bir testle doğrularsınız. Aynı koltuk için ardışık iki `POST /biletler` isteği gönderen (senkron, kolay) test şöyledir:

```python
def test_ardisik_rezervasyon_409(client):
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)

    ilk = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    )
    assert ilk.status_code == 201

    ikinci = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Mehmet Demir"}
    )
    assert ikinci.status_code == 409
    assert "satılmış" in ikinci.json()["detail"]

    biletler = client.get("/biletler").json()
    assert len(biletler) == 1
```

Dene-Çalıştır

Bu opsiyonel testte, gerçekten eşzamanlı iki isteği `ThreadPoolExecutor` ile tetikleyen, yalnızca birinin `201`, diğerinin `409` aldığını doğrulayan bir test bulunur — yukarıdaki ardışık testten daha kırılgandır (zamanlama duyarlı), çünkü gerçek bir yarış durumunu simüle etmeye çalışır. Kurulum bölümü kendi dosya tabanlı SQLite veritabanını (diğer testlerin paylaştığı bellek-içi `:memory:` veritabanının aksine) kurar — gerçek bir SQLite dosya kilidinin iki ayrı bağlantı arasında nasıl çalıştığını sınamak için:

```python
db_yolu = tmp_path / "esz_zamanli_test.db"
engine = create_engine(
    f"sqlite:///{db_yolu}",
    connect_args={"check_same_thread": False, "timeout": 30},
)
sqlite_yazma_kilidi_uygula(engine)
Base.metadata.create_all(engine)
session_fabrikasi = sessionmaker(bind=engine)

def get_db_gecersiz_kil():
    db = session_fabrikasi()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = get_db_gecersiz_kil
```

Ardından iki isteği gerçekten aynı anda göndermek için bir

`ThreadPoolExecutor` kullanılır — iki iş parçacığı `rezervasyon_yap` fonksiyonunu eşzamanlı çağırır, her ikisinin cevabı beklenir:

```
with TestClient(app) as istemci:
    _, koltuk = _etkinlik_ve_koltuk_olustur(istemci)

    def rezervasyon_yap(kullanici_adi: str):
        return istemci.post(
            "/biletler",
            json={"koltukId": koltuk["id"], "kullaniciAdi": kullanici_adi
                },
        )

    with ThreadPoolExecutor(max_workers=2) as havuz:
        gorev_1 = havuz.submit(rezervasyon_yap, "Ayşe Yılmaz")
        gorev_2 = havuz.submit(rezervasyon_yap, "Mehmet Demir")
        cevap_1 = gorev_1.result()
        cevap_2 = gorev_2.result()
```

Son olarak iki cevabın durum kodları sıralanıp karşılaştırılır — Ünite 4'ün tek-transaction korumasının doğru çalıştığının kanıtı, bunlardan tam olarak birinin `201`, diğerinin `409` olmasıdır:

```
durum_kodlari = sorted([cevap_1.status_code, cevap_2.status_code])
assert durum_kodlari == [201, 409]
```

Tüm test paketini çalıştırdığınızda (Hf.6-7'nin testleri + bu haftanın yeni transaction/cascade/sayfalama testleri) hatasız geçer:

Terminal

```
$ python -m pytest -q
```

Beklenen Çıktı

```
................                                         [100%]
16 passed in 0.19s
```

8.4.8 `Depends`'e VT oturumu gelir: Hf.6'nın ileri bağı kapanır

Hf.6 Ünite 4'te `get_depo`'nun "Hf.8'de `get_db_session()` bu yer tutucuyu değiştirecek" notunu bırakmıştınız. Bu söz şimdi gerçekleşir:

```
def get_depo(db: Session = Depends(get_db)) -> Depo:

    return Depo(db)
```

Bu bir zincirleme bağımlılıktır: `get_depo`'nun kendisi de bir `Depends` parametresi alır, `get_db`'ye bağlı — bir `Depends`'in içinde başka bir `Depends` kullanılabilir. Rota fonksiyonlarındaki `depo: Depo = Depends(get_depo)` yazımı Hf.6-7'den tek bir karakter değişmeden kalır; değişen yalnızca `get_depo`'nun içinde artık bir Python `dict` yerine bir veritabanı `Session` kullanılmasıdır — "aynı mekanizma, farklı bağımlılık" sözü tam olarak budur.

3 • Katılım **⌨ Şimdi Siz Deneyin**

`test_transaction.py`'de aynı koltuk için ardışık iki `POST /biletler` isteği gönderin; ikinci isteğin durum kodunu ve gövdesini inceleyin.

Geribildirim. ilk istek `201` döner, koltuk dolu olur; ikinci istek koltuğun zaten dolu olduğunu transaction içinde görür ve `409` ile reddedilir — hiçbir ikinci bilet kaydedilmez, veritabanı tutarlı kalır.

Kendinizi Sınayın 8.4

Neden `400 Bad Request` değil `409 Conflict` + `IntegrityError` yakalama kullanılır; "kontrol et + güncelle + yaz" adımlarının tek bir transaction/commit sınırında olması neden gereklidir?

5 • İzleme **Aktarım — Başka Nerede?**

Bu haftaki tek-transaction disiplini Hf.13'ün WebSocket bildirimlerinde ("koltuk durumu değiştiğinde herkese haber ver") ve Hf.15'in katmanlı mimarisinde (servis katmanının transaction sınırlarını yönetmesi) aynı ilkeyle sürecek. Etkinlik/Bilet Sisteminiz artık Hf.6-7'nin API sözleşmesini hiç değiştirmeden, gerçekten kalıcı ve gerçekten eşzamanlı-güvenli bir depoya sahip.

Bölüm Özeti

Özet

Bu hafta Hf.6-7'nin "sunucunun hafızası hâlâ geçici" sınırını ortadan kaldırdınız. İlk ünitede SQLAlchemy 2.x'in bildirimsel model biçimini (`DeclarativeBase`, `Mapped`, `mapped_column`) öğrendiniz, bir `engine/Session/get_db` deseni kurup `bilet.db` dosyasında `create_all` ile ilk tablolarınızı oluşturdunuz ve ORM modeli (`EtkinlikDB`) ile Pydantic modelinin (`Etkinlik`) neden ayrı kaldığını, aralarındaki köprüyü (`model_validate(from_attributes=True)`) gördünüz. İkinci ünitede `ForeignKey/relationship` ile Etkinlik–Koltuk–Bilet ilişkilerini kurdunuz, tembel/istekli yükleme farkını ve N+1 sorununu tanıdınız, bir etkinlik silindiğinde satılmış bilet varsa `409`, yoksa kademeli (cascade) silme kuralını uyguladınız. Üçüncü ünitede `select()/where/order_by/limit/offset` ile sayfalanmış sorgular yazdınız, `echo=True` ile ORM'in ürettiği gerçek SQL'i YG II'nin elle yazdığınız SQL'iyle karşılaştırdınız ve `text()` kaçış kapısını tanıdınız. Dördüncü ünitede bir rezervasyonun "kontrol et + dolu yap + yaz" üç adımının tek bir transaction'da nasıl korunduğunu, SQLite'ın `BEGIN IMMEDIATE` yazma kilidini, kısmi `UNIQUE` indeksin son güvence katmanı olarak nasıl çalıştığını ve bunu `pytest` ile nasıl test ettiğinizi gördünüz. Sonuçta Etkinlik/Bilet Sisteminiz Hf.6-7'nin API sözleşmesini hiç değiştirmeden, kalıcı ve eşzamanlı-güvenli bir depoya kavuştu. Gelecek hafta Hafta 9 Ara Değerlendirme'dir — yeni konu yok, Hf.1-8'i sentezleyen bir arabölüm.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — `EtkinlikDB` ile `Etkinlik` neden aynı sınıf değil). `EtkinlikDB` bir veritabanı satırının Python temsilidir — bir `Session`'a bağlıdır ve ORM'e özgü davranışlar (tembel yükleme) taşır; `Etkinlik` ise dışa dönük, veritabanından tamamen bağımsız bir JSON sözleşmesidir — ikisini birleştirmek katmanları karıştırır. `model_validate(from_attributes=True)` bir `EtkinlikDB` nesnesinin özniteliklerini okuyup bir `Etkinlik` Pydantic nesnesi üreterek bu iki katman arasında bir köprü kurar — rota fonksiyonlarının dönüş tipi hâlâ `Etkinlik`'tir, bu köprü sayesinde Hf.6-7'nin rota imzaları hiç değişmez.

Soru (Ünite 2 — N+1 sorunu ve çözümü). `GET /etkinlikler` isteğinde her etkinliğin koltuk sayısını ayrı ayrı bir sorguyla çekerseniz (tembel yükleme, N etkinlik için N ek sorgu), "N+1 sorunu" oluşur — 1 ana sorgu + N ilişki sorgusu, etkinlik sayısı arttıkça sorgu sayısı doğrusal olarak artar. `selectinload(EtkinlikDB.koltuklar)` ile tüm ilişkiler tek bir ek sorguda önceden çekilerek (istekli/eager yükleme) sorgu sayısı sabit tutulur — etkinlik sayısı ne olursa olsun toplam sorgu sayısı ikide kalır (1 ana + 1 istekli).

Soru (Ünite 3 — `echo=True` ile ORM SQL'inin farklı görünmesi bir sorun mu). sorun değildir — ORM sorguyu kendi iç temsilinden (Python nesne modelinden) üretir, insan gibi "en kısa/en okunaklı" SQL yazmaya çalışmaz; sütun adlarının tam tablo adıyla öneklenmesi, değerlerin parametreli `?` yer tutucularıyla ayrı gönderilmesi gibi biçimsel farklar olabilir, ama fonksiyonel olarak aynı sonucu verir. Bu yüzden performans-kritik/karmaşık sorgularda `text()` ile ham SQL yazmak bazen tercih edilir — ama bu bir zorunluluk değil, bir seçenektir.

Soru (Ünite 4 — `409` neden `400` değil; tek transaction neden gerekli). İstek format olarak geçerlidir (422/400 tetiklenmez), ama sunucudaki mevcut durumla (koltuk zaten satılmış) çelişir — bu `409`'un tanımıdır. Adımlar ayrı ayrı `commit` edilseydi (kontrol et → COMMIT, güncelle → COMMIT), iki commit arasındaki boşlukta başka bir isteğin araya girip aynı koltuğu alması mümkün olurdu — tek transaction bu boşluğu tamamen kapatır, "ya hep ya hiç" garantisi verir; SQLite'ın `BEGIN IMMEDIATE` yazma kilidi bu garantiyi gerçek bir veritabanı seviyesinde uygular, kısmi `UNIQUE` indeks ise uygulama kontrolü atlansa bile son bir güvence katmanı sağlar.